

Quiz 7 Solution

Q1. Why is recursion more preferred over iterative practices?

- a) Recursion provides a more concise and elegant solution, especially in divide-and-conquer problems, by breaking down complex problems into smaller, easier-to-solve subproblems.
- b) Recursion uses less memory because it avoids the overhead of maintaining loop counters, and therefore is always more memory-efficient than iterative methods.
- c) Recursion guarantees faster execution times, as recursive calls can be optimized by compilers to reduce the need for additional iterations in iterative methods.
- d) Recursion eliminates the need for loops entirely, which improves efficiency by avoiding the potential errors that can arise from managing loop conditions manually.

Correct: a)

Q2. What is the maximum *recursion depth* and *function calls* to solve this problem?

Note: Assume N to be problem size and K to be recursive depth.

```
double breakAndSolve(double problem){  
    if(problem < 1) return 0;  
  
    return breakAndSolve(problem/2) + breakAndSolve(problem/2);  
}
```

- a) $\log_2(N)$ and 2^k
- b) 2^k and $\log_2(N)$
- c) 2^N and $\log_2(K)$
- d) $\log_2(K)$ and 2^N

Correct: a)

Q3. In-order traversal in a Binary Tree means:

- a) Left $\rightarrow$ Root $\rightarrow$ Right
- b) Right $\rightarrow$ Root $\rightarrow$ Left
- c) Root $\rightarrow$ Left $\rightarrow$ Right
- d) Left $\rightarrow$ Right $\rightarrow$ Root

Correct: a)

Q4. Consider the function:

```
void fun(int n) {  
    if (n <= 0) return;  
    printf("%d ", n);  
    fun(n - 1);  
}  
  
int main(){  
    fun(4);  
}
```

How many times is fun() called, including the initial call?

- a) 3
- b) 4
- c) 5
- d) Infinite

Correct: c)

Q5. Which of the following techniques can improve the efficiency of backtracking algorithms?

- a) Sorting the input before the search begins.
- b) Avoiding recursion altogether and using iteration.
- c) Using memoization to store results of previously computed solutions.
- d) Limiting the recursion depth to a fixed value.

Correct: c)

Q6. Which of the following is a key drawback of using recursion in a function?

- a) It requires less memory compared to iteration.
- b) Recursive calls consume additional stack space for each function call.
- c) It always leads to a more optimal solution compared to iterative solutions.
- d) It never causes infinite loops.

Correct: b)

Q7. In backtracking algorithms, pruning refers to:

- a) Cutting down the size of the input data to speed up the search.
- b) Terminating the search immediately once a valid solution is found.
- c) Stopping the exploration of certain branches of the solution tree that cannot lead to a valid solution.
- d) Iterating through all possible solutions without any termination condition.

Correct: c)

Q8. Which of the following is not a valid termination condition (base case) for a recursive function?

- a) When the problem is reduced to a trivial case that can be solved directly.
- b) When the recursion depth exceeds a certain limit.
- c) When the function recurses on the same input repeatedly.
- d) When the input size becomes zero or negative.

Correct: c)

Q9. Why is recursive binary search considered comparatively better than iterative binary search in certain scenarios?

- a) Recursive binary search has a lower time complexity than iterative binary search, making it faster in large datasets.
- b) Recursive binary search is easier to implement, especially for problems where you need to keep track of multiple parameters or states.
- c) Iterative binary search has a higher space complexity due to the overhead of maintaining function calls in the recursion stack.
- d) Recursive binary search uses fewer comparisons to find the target element compared to iterative binary search.

Correct: b)

Q10. Complete the following piece of code to generate a Power set of a given set.

Note: Power set refers to every possible subset that can be generated from a given set. i.e. {1,2,3} power set will be {{}, {1}, {2}, {3}, {1,2}, {1,3}, {2,3}, {1,2,3}}

```
void populatePowerSet(vector<int>& nums,int index,vector<int>& sequence,vector<vector<int>>& result){
    if(index == nums.size()) {
        result.push_back(sequence);
        return;
    }
    

?


}
```

```
vector<vector<int>> powerSet(vector<int> nums){
    vector<int> singleSet;
    vector<vector<int>> result;
    populatePowerSet(nums,0,singleSet,result);
    return result;
}
```

a)

```
sequence.push_back(nums[index]);

populatePowerSet(nums,index+1,sequence,result);

sequence.pop_back();

populatePowerSet(nums,index+1,sequence,result);
```

b)

```
sequence.push_back(nums[index]);

populatePowerSet(nums,index+1,sequence,result);

sequence.pop_back();

populatePowerSet(nums,index,sequence,result);
```

c)

```
sequence.push_back(nums[index]);

populatePowerSet(nums,index+1,sequence,result);

populatePowerSet(nums,index+1,sequence,result);
```

d)

```
sequence.push_back(nums[index]);

populatePowerSet(nums,index+1,sequence,result);

populatePowerSet(nums,index,sequence,result);
```

Correct: a)

Q11. What is the correct base case for a recursive factorial function `int factorial(int n)`?

- a) `if (n == 0) return 0;`
- b) `if (n == 1) return 1;`
- c) `if (n == 0) return 1;`
- d) `if (n == 2) return 2;`

Correct: c)

Q12. In the following code, what is the value of `n` during the 3rd recursive call (counting the original call as 1st)?

```
void print(int n) {  
    if (n == 0) return;  
    cout << n << " ";  
    print(n-1);  
}
```

```
int main() {  
    print(5);  
}
```

- a) `n = 5`
- b) `n = 4`
- c) `n = 3`
- d) `n = 2`

Correct: c)

Q13. Suppose you have a function:

```
int sumArray(int arr[], int size, int index) {  
    if (_____) return 0;  
    return arr[index] + sumArray(arr, size, index + 1);  
}
```

Which condition correctly identifies the base case in `sumArray`?

- a) `if (index == 0)`
- b) `if (index == size)`
- c) `if (index == size - 1)`
- d) `if (index < 0)`

Correct: b)

Q14. Given this code:

```
int mystery(int n) {  
    if (n == 0) return 0;  
    return n + mystery(n-2);  
}
```

What value is returned by `mystery(4)`?

- a) 6
- b) 4
- c) 3
- d) 7

Correct: a)

Q15. Which statement about recursive functions is always true?

- a) Recursive functions must call themselves without any conditions.
- b) Recursive functions must have a base case to terminate recursion.
- c) Recursive functions must use loops internally to work.
- d) Recursive functions must return immediately without any processing.

Correct: b)

**Q16. In the code below, how many total `explore` function calls are made when `level = 3`?
(Including the initial call.)**

```
void explore(int level) {  
    if (level == 0) return;  
    explore(level-1);  
    explore(level-1);  
}
```

- a) 6
- b) 7
- c) 8
- d) 15

Correct: d)

Q17. What happens if a recursive function has no base case?

- a) It executes normally and stops automatically after a few calls.
- b) It causes infinite recursion, leading to a stack overflow.
- c) It only recurses twice and stops.
- d) It behaves like an iterative function.

Correct: b)

Q18. In the following subset sum recursion, what should happen if `current\_sum` becomes greater than `target`?

```
bool subset_sum(vector<int>& arr, int index, int current_sum, int target) {  
    if (current_sum > target) -----;  
    if (current_sum == target) return true;  
    if (index == arr.size()) return false;  
    return subset_sum(arr, index + 1, current_sum + arr[index], target) ||  
        subset_sum(arr, index + 1, current_sum, target);  
}
```

- a) Return false immediately.
- b) Continue to the next element.
- c) Reset current_sum to 0.
- d) Start recursion from index 0 again.

Correct: a)

Q19. Which situation would naturally require recursion and is hard to solve cleanly with only loops?

- a) Finding the maximum in an array.
- b) Traversing a binary tree.
- c) Reversing an array.
- d) Printing numbers from 1 to 10.

Correct: b)

Q20. When recursively searching for an integer value inside a binary tree node structure, which conditions should you check?

- a) If node is NULL, return false; if node is greater then, search left only; if node is less then, search right only; if node is equal, return true.
- b) If node has two children, if node is a leaf, then search left only.
- c) If node's left child is greater, then search left only.
- d) If node is NULL, always return true, and check only the right child.

Correct: a)